University of Bamberg

Chair of Explainable Machine Learning

Deep Learning Exercise

Assignment 1 - Writeup

Tabish Izhar

*ba*078794



Collaborators

None

1 Derivatives [8 points]

1.1 Partial Derivative [2 points]

Given:

$$f(x) = \begin{pmatrix} e^{x_1} \sin(x_3) \\ \sqrt{x_2 + x_3} \end{pmatrix} \qquad \text{with } f : \mathbb{R}^3 \mapsto \mathbb{R}^2 \tag{1}$$

Calculate the matrix of partial derivatives $\frac{\partial f}{\partial x}(x)$.
Solution:
The Jacobian $\frac{\partial f}{\partial x}(x) \in \mathbb{R}^{2\times 3}$ has one row per output f_i and one column per input x_j, i.e. entry (i, j) is $\frac{\partial f_i}{\partial x_j}$:

$$\frac{\partial f}{\partial x}(x) = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \frac{\partial f_1}{\partial x_3} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \frac{\partial f_2}{\partial x_3} \end{pmatrix} = \begin{pmatrix} e^{x_1} \sin(x_3) & 0 & e^{x_1} \cos(x_3) \\ 0 & \frac{1}{2\sqrt{x_2 + x_3}} & \frac{1}{2\sqrt{x_2 + x_3}} \end{pmatrix}$$

1.2 Derivative of the Sigmoid Function [4 points]

a) The sigmoid function is defined as:

$$\sigma(x) = \frac{1}{1+\mathrm{e}^{-x}} \qquad \text{with } \sigma : \mathbb{R}^1 \mapsto \mathbb{R}^1 \tag{2}$$

Calculate $\sigma'(x)$. The solution should be simplified as much as possible and include the sigmoid function somehow...

Solution:

Writing $\sigma(x) = (1+\mathrm{e}^{-x})^{-1}$ and applying the chain rule:

$$\begin{aligned}
\sigma'(x) &= -(1+\mathrm{e}^{-x})^{-2} \cdot (-\mathrm{e}^{-x}) = \frac{\mathrm{e}^{-x}}{(1+\mathrm{e}^{-x})^2} \\
&= \frac{1}{1+\mathrm{e}^{-x}} \cdot \frac{(1+\mathrm{e}^{-x})-1}{1+\mathrm{e}^{-x}} = \frac{1}{1+\mathrm{e}^{-x}} \cdot \left(1 - \frac{1}{1+\mathrm{e}^{-x}}\right) \\
&= \sigma(x)\Big(1-\sigma(x)\Big)
\end{aligned}$$

b) With $a : \mathbb{R}^1 \mapsto \mathbb{R}^1$, we can define a function $f(x)$ as:

$$f(x) = \begin{pmatrix} a(x_1) \\ \vdots \\ a(x_n) \end{pmatrix} \tag{3}$$

Calculate the matrix of partial derivatives $\frac{\partial f}{\partial x}(x)$.

Solution:

The i-th output is $f_i(x) = a(x_i)$, which depends only on x_i. Hence

$$\frac{\partial f_i}{\partial x_j} = \begin{cases} a'(x_i) & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases}$$

and therefore

$$\frac{\partial f}{\partial x}(x) = \begin{pmatrix} a'(x_1) & 0 & \cdots & 0 \\ 0 & a'(x_2) & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & a'(x_n) \end{pmatrix} = \operatorname{diag}\Big(a'(x_1), \ldots, a'(x_n)\Big) \in \mathbb{R}^{n \times n}$$

c) Replace a in the function above with the sigmoid function σ, such as:

$$f(x) = \begin{pmatrix} \sigma(x_1) \\ \vdots \\ \sigma(x_n) \end{pmatrix} \tag{4}$$

Calculate the matrix of partial derivatives $\frac{\partial f}{\partial x}(x)$.

Solution:

Using b) with $a = \sigma$ and the result $\sigma'(x) = \sigma(x)(1 - \sigma(x))$ from a):

$$\frac{\partial f}{\partial x}(x) = \begin{pmatrix} \sigma(x_1)\big(1 - \sigma(x_1)\big) & 0 & \cdots & 0 \\ 0 & \sigma(x_2)\big(1 - \sigma(x_2)\big) & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma(x_n)\big(1 - \sigma(x_n)\big) \end{pmatrix}$$

d) What property has the resulting matrix $\frac{\partial f}{\partial x}(x)$ of task c?

Solution:

The matrix is a **diagonal matrix**: all off-diagonal entries are 0, because each output $\sigma(x_i)$ depends only on its own input x_i (the sigmoid is applied element-wise). Consequently it is also square ($n \times n$) and symmetric. Moreover, since $\sigma(x_i) \in (0,1)$, every diagonal entry satisfies $0 < \sigma(x_i)(1-\sigma(x_i)) \leq \frac{1}{4}$, so the matrix is positive definite (and invertible).

1.3 Weight Matrix [2 points]

Let $\mathbf{W} \in \mathbb{R}^{m \times n}, \mathbf{x} \in \mathbb{R}^n$ and $\mathbf{y} = \mathbf{W}\mathbf{x}$.

a) Calculate the partial derivative $\frac{\partial \mathbf{y}}{\partial \mathbf{x}}$.

Solution:

Component-wise, $y_k = \sum_{l=1}^{n} w_{kl} x_l$ for $k \in \{1, \ldots, m\}$. Hence

$$\frac{\partial y_k}{\partial x_j} = w_{kj} \qquad \Rightarrow \qquad \frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \mathbf{W} \in \mathbb{R}^{m \times n}$$

b) Calculate $\frac{\partial y_k}{\partial w_{ij}}$ for $k \in \{1,...,m\}$, $i \in \{1,...,m\}$ and $j \in \{1,...,n\}$.

Solution:

From $y_k = \sum_{l=1}^{n} w_{kl}\, x_l$, the weight w_{ij} only appears in the sum for y_i (i.e. when $k = i$, $l = j$). Therefore

$$\frac{\partial y_k}{\partial w_{ij}} = \begin{cases} x_j & \text{if } k = i \\ 0 & \text{if } k \neq i \end{cases} = \delta_{ki}\, x_j$$

where δ_{ki} is the Kronecker delta. In vector form, $\frac{\partial \mathbf{y}}{\partial w_{ij}} = x_j\, \mathbf{e}_i \in \mathbb{R}^m$, with $\mathbf{e}_i$ the i-th standard basis vector (all zeros except a 1 at position i).

c) Let $\mathbf{A} \in \mathbb{R}^{1\times m}$. Simplify

$$\widehat{w}_{ij} = \mathbf{A}\frac{\partial \mathbf{y}}{\partial w_{ij}} \tag{5}$$

Solution:

Using b), $\frac{\partial \mathbf{y}}{\partial w_{ij}} = x_j\,\mathbf{e}_i$. Multiplying the row vector $\mathbf{A} = (A_1, \ldots, A_m)$ with it:

$$\widehat{w}_{ij} = \mathbf{A}\,\frac{\partial \mathbf{y}}{\partial w_{ij}} = \sum_{k=1}^{m} A_k\,\frac{\partial y_k}{\partial w_{ij}} = \sum_{k=1}^{m} A_k\,\delta_{ki}\,x_j = A_i\,x_j$$

So $\widehat{w}_{ij}$ is simply the i-th entry of $\mathbf{A}$ times the j-th entry of $\mathbf{x}$ (collected over all i, j this gives the outer product $\widehat{\mathbf{W}} = \mathbf{A}^\top\mathbf{x}^\top \in \mathbb{R}^{m\times n}$).

2 Linear Regression [2 points]

The loss function $\mathcal{L}$ of a Linear Regression model with a scalar input x and scalar output y is given by:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} (w_0 + w_1 x_i - y_i)^2 \tag{6}$$

In this simple 1D case, we are fitting a straight line to the dataset. This line is determined by its offset along the y-axis (w_0) and its slope (w_1). Compute the partial derivative of the loss w.r.t. the y-offset $\left(\frac{\partial \mathcal{L}}{\partial w_0}\right)$ and the partial derivative of the loss w.r.t. the slope $\left(\frac{\partial \mathcal{L}}{\partial w_1}\right)$.

Solution:

Let $e_i = w_0 + w_1 x_i - y_i$ denote the error of sample i, so that $L = \frac{1}{N} \sum_{i=1}^{N} e_i^2$. Since the derivative of a sum is the sum of the derivatives, and by the chain rule $\frac{\partial}{\partial w} e_i^2 = 2\, e_i \frac{\partial e_i}{\partial w}$, we get

$$\frac{\partial L}{\partial w} = \frac{1}{N} \sum_{i=1}^{N} 2\, e_i \frac{\partial e_i}{\partial w}.$$

The inner derivatives are

$$\frac{\partial e_i}{\partial w_0} = 1, \qquad \frac{\partial e_i}{\partial w_1} = x_i.$$

Therefore

$$\frac{\partial L}{\partial w_0} = \frac{2}{N} \sum_{i=1}^{N} (w_0 + w_1 x_i - y_i),$$

$$\frac{\partial L}{\partial w_1} = \frac{2}{N} \sum_{i=1}^{N} (w_0 + w_1 x_i - y_i)\, x_i.$$

3 Optimization [15 points + 10 points bonus]

3.1 Level Surface Set [10 points]

Let $f(\mathbf{x})$ be a function, where $\mathbf{x} \in \mathbb{R}^d$. The *level surface* of a function is the set of points with the same value of function, i.e.

$$L_c = \{\mathbf{x} \mid f(\mathbf{x}) = c\} \tag{7}$$

for some $c \in \mathbb{R}$.

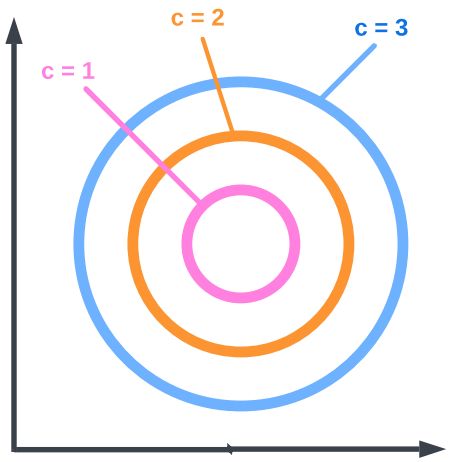


Figure 1: 2D example for level surfaces for three different values.

Figure 1 shows level surfaces for different values of c for a 2-dimensional case. The points on the perimeter of each curve illustrate the set of points sharing the same value, i.e. $f(x_1, x_2) = c$.

Recall that a gradient vector is defined as $\nabla f = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, ..., \frac{\partial f}{\partial x_d}\right]$.

Consider a specific point $\mathbf{x}_0 \in \mathbb{R}^d$. Let us denote the level surface passing through this point by $L_{f(\mathbf{x_0})}$. Let us denote the gradient vector at $\mathbf{x_0}$ by ∇f_0.

Now, consider an arbitrary curve $\mathbf{r}(t) = [x_1(t); x_2(t); \ldots, x_d(t)]$ (parameterized by a scalar t) that passes through $\mathbf{x}_0$, i.e. $\mathbf{r}(t_0) = \mathbf{x_0}$ for some $t_0 \in \mathbb{R}$. Furthermore, the curve $\mathbf{r}(t)$ lies within the level surface passing through $\mathbf{x}_0$, i.e. $\mathbf{r}(t) \in L_{f(\mathbf{x_0})}$, $\forall t$.

Recall that the tangent to a curve is defined as $\frac{\partial \mathbf{r}}{\partial t}$.

With all that context and notation behind us, now comes the 'fun' part. Show that ∇f_0 is orthogonal to the tangent of $\mathbf{r}(t)$ at t_0.

Now, please describe in non-technical language what you have just proven. Why might we care about this in the context of deep learning?

Solution:
Since the curve lies completely within the level surface through x_0, the function value along the curve never changes:

$$f(r(t)) = f(x_0) = c \qquad \forall t.$$

We differentiate both sides with respect to t. The right-hand side is a constant, so its derivative is 0. For the left-hand side we apply the multivariable chain rule, since every coordinate x_i depends on t:

$$\frac{d}{dt} f(r(t)) = \sum_{i=1}^{d} \frac{\partial f}{\partial x_i}\Big(r(t)\Big) \frac{dx_i}{dt}(t) = \left[\frac{\partial f}{\partial x_1}, \ldots, \frac{\partial f}{\partial x_d}\right] \cdot \left[\frac{dx_1}{dt}, \ldots, \frac{dx_d}{dt}\right]^\top = \nabla f\Big(r(t)\Big) \cdot \frac{\partial r}{\partial t}(t).$$

Hence

$$\nabla f\Big(r(t)\Big) \cdot \frac{\partial r}{\partial t}(t) = 0 \qquad \forall t.$$

Evaluating at $t = t_0$, where $r(t_0) = x_0$, gives

$$\nabla f_0 \cdot \frac{\partial r}{\partial t}(t_0) = 0.$$

A dot product of zero means that the two vectors are orthogonal. Since r was an arbitrary curve in the level surface, ∇f_0 is orthogonal to every tangent direction of the level surface at x_0. ■

Non-technical explanation: Imagine standing on a hill and walking along a path on which your height never changes (a contour line on a map). The gradient points in the direction in which the hill rises most steeply. We have shown that this direction of steepest ascent is always at a right angle to the "flat" path. Intuitively: if the gradient had any component along the path, walking along the path would change the height, which contradicts the path being at constant height.

Relevance for deep learning: In training, the "hill" is the loss function over the space of model parameters, and its level surfaces are sets of parameters that give the same loss. Gradient descent moves in the direction $-\nabla L$, i.e. perpendicular to the level surfaces of the loss. This is the direction of steepest decrease, so a small step reduces the loss as much as locally possible, whereas a step along a level surface would not change the loss at all. The result also explains typical behaviour of gradient descent: in long, narrow valleys the level surfaces are elongated, so the gradient points across the valley instead of along it, causing zig-zagging and slow progress. This motivates improved optimizers such as momentum or Adam and normalization of the inputs.

3.2 Bonus: Relation Gradient - Local Minimum [5 points]

Consider a (not necessarily convex) differentiable function $g : \mathbb{R}^n \to \mathbb{R}$. g has a local minimum at some $\mathbf{w}^t$ if there exists some $\gamma > 0$ such that for all $\mathbf{w} \in \mathbb{R}^n$, $\|\mathbf{w}^t - \mathbf{w}\|_2 < \gamma \Rightarrow g(\mathbf{w}^t) \leq g(\mathbf{w})$.
Prove that if g has a local minimum at some $\mathbf{w}^t$ then the gradient at $\mathbf{w}^t = 0$, and that the converse is not necessarily true.
Solution:
Part 1: local minimum $\Rightarrow \nabla g(w_t) = 0$ (proof by contradiction).
Assume g has a local minimum at w_t, but $v := \nabla g(w_t) \neq 0$. Consider the one-dimensional function along the negative gradient direction

$$h(\varepsilon) = g(w_t - \varepsilon v), \qquad \varepsilon \in \mathbb{R}.$$

Since g is differentiable, h is differentiable, and by the chain rule

$$h'(0) = \nabla g(w_t)^\top (-v) = -v^\top v = -\|v\|_2^2 < 0.$$

By the definition of the derivative, $h'(0) = \lim_{\varepsilon \to 0} \frac{h(\varepsilon) - h(0)}{\varepsilon} < 0$, so there exists $\varepsilon_0 > 0$ such that $h(\varepsilon) < h(0)$ for all $0 < \varepsilon < \varepsilon_0$. Choose such an ε that additionally satisfies $\varepsilon < \gamma / \|v\|_2$ and set $w = w_t - \varepsilon v$. Then

$$\|w_t - w\|_2 = \varepsilon \|v\|_2 < \gamma \qquad \text{and} \qquad g(w) = h(\varepsilon) < h(0) = g(w_t),$$

which contradicts w_t being a local minimum. Hence $\nabla g(w_t) = 0$. ■
Part 2: the converse is not necessarily true (counterexample).
Consider $g : \mathbb{R}^n \to \mathbb{R}$, $g(w) = w_1^3$, at $w_t = 0$. Its gradient is $\nabla g(w) = (3w_1^2, 0, \ldots, 0)^\top$, so $\nabla g(0) = 0$. However, for every $\gamma > 0$ the point $w = (-\frac{\gamma}{2}, 0, \ldots, 0)^\top$ satisfies $\|w - 0\|_2 = \frac{\gamma}{2} < \gamma$ and

$$g(w) = -\frac{\gamma^3}{8} < 0 = g(0).$$

Thus 0 is not a local minimum although the gradient vanishes there. (Another example is the saddle point of $g(w) = w_1^2 - w_2^2$ at 0.) Hence a zero gradient is necessary, but not sufficient, for a local minimum.

3.3 Bonus: Relation Gradient - Global Minimum [5 points]

Prove that if a differentiable function $g : \mathbb{R}^n \to \mathbb{R}$ is convex and the gradient at some $\mathbf{w}^*$ is 0, then $\mathbf{w}^*$ is the global minimum of g.
Solution:
By the definition of convexity, for all $a, b \in \mathbb{R}^n$ and $\lambda \in [0, 1]$:

$$g\Big(\lambda b + (1-\lambda)a\Big) \leq \lambda\, g(b) + (1-\lambda)\, g(a).$$

Let $w \in \mathbb{R}^n$ be arbitrary and choose $a = w^*$, $b = w$. Since $\lambda w + (1-\lambda)w^* = w^* + \lambda(w - w^*)$, this gives

$$g\Big(w^* + \lambda(w - w^*)\Big) \leq g(w^*) + \lambda\Big(g(w) - g(w^*)\Big).$$

Subtracting $g(w^*)$ and dividing by $\lambda \in (0, 1]$ yields

$$\frac{g\Big(w^* + \lambda(w - w^*)\Big) - g(w^*)}{\lambda} \leq g(w) - g(w^*).$$

Letting $\lambda \to 0^+$, the left-hand side becomes the directional derivative of g at w^* in direction $w - w^*$, which, since g is differentiable, equals $\nabla g(w^*)^\top (w - w^*)$. Hence we obtain the first-order condition of convexity

$$g(w) \geq g(w^*) + \nabla g(w^*)^\top (w - w^*).$$

Using $\nabla g(w^*) = 0$, this simplifies to

$$g(w) \geq g(w^*).$$

Since w was arbitrary, $g(w^*) \leq g(w)$ for all $w \in \mathbb{R}^n$, i.e. w^* is a global minimum of g.
■

3.4 Gradient of the Softmax Function [5 points]

We will now consider the softmax function $\mathbf{s}(\mathbf{z})$, which takes a vector input $\mathbf{z}$ and outputs a vector whose i-th entry s_i is

$$s_i = \frac{e^{z_i}}{\sum_k e^{z_k}} \tag{8}$$

The input vector $\mathbf{z}$ to $\mathbf{s}(\cdot)$ is sometimes called the "logits", which just means the unscaled output of previous layers. Derive the gradient of $\mathbf{s}$ with respect to the logits, i.e. derive $\frac{\partial \mathbf{s}}{\partial \mathbf{z}}$.

Solution:

Let $S = \sum_k \mathrm{e}^{z_k}$, so that $s_i = \frac{\mathrm{e}^{z_i}}{S}$. We need two helper derivatives:

$$\frac{\partial \mathrm{e}^{z_i}}{\partial z_j} = \begin{cases} \mathrm{e}^{z_i} & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases}, \qquad \frac{\partial S}{\partial z_j} = \mathrm{e}^{z_j}.$$

Using the quotient rule $\left(\frac{u}{v}\right)' = \frac{u'v - uv'}{v^2}$ with $u = \mathrm{e}^{z_i}$ and $v = S$:

Case $i = j$:

$$\frac{\partial s_i}{\partial z_i} = \frac{\mathrm{e}^{z_i} S - \mathrm{e}^{z_i}\mathrm{e}^{z_i}}{S^2} = \frac{\mathrm{e}^{z_i}}{S} - \left(\frac{\mathrm{e}^{z_i}}{S}\right)^2 = s_i - s_i^2 = s_i(1 - s_i).$$

Case $i \neq j$:

$$\frac{\partial s_i}{\partial z_j} = \frac{0 \cdot S - \mathrm{e}^{z_i}\mathrm{e}^{z_j}}{S^2} = -\frac{\mathrm{e}^{z_i}}{S} \cdot \frac{\mathrm{e}^{z_j}}{S} = -s_i s_j.$$

Both cases can be combined using the Kronecker delta δ_{ij} (1 if $i = j$, 0 otherwise):

$$\frac{\partial s_i}{\partial z_j} = s_i\left(\delta_{ij} - s_j\right).$$

In matrix form, with $s = (s_1, \ldots, s_n)^\top$:

$$\frac{\partial s}{\partial z} = \operatorname{diag}(s) - s\,s^\top = \begin{pmatrix} s_1(1-s_1) & -s_1 s_2 & \cdots & -s_1 s_n \\ -s_2 s_1 & s_2(1-s_2) & \cdots & -s_2 s_n \\ \vdots & \vdots & \ddots & \vdots \\ -s_n s_1 & -s_n s_2 & \cdots & s_n(1-s_n) \end{pmatrix}.$$

In contrast to the element-wise sigmoid (task 1.2), this Jacobian is not diagonal, since every output s_i depends on all logits through the shared normalization S. The matrix is symmetric.

4 Directed Acyclic Graph [10 points]

One important property for a feed-forward network, as discussed in the lectures, is that it must be a directed acyclic graph (DAG). A *DAG is a directed graph that contains no directed cycles*. We will study some of its properties in this question.

Let's define a graph $G = (V, E)$ in which V is the set of all nodes as $\{v_1, v_2, ..., v_i, ...v_n\}$ and E is the set of edges $E = \left\{e_{i,j} = (v_i, v_j) \mid v_i, v_j \in V\right\}$.

A *topological order of a directed graph* $G = (V, E)$ is an ordering of its nodes as $\{v_1, v_2, ..., v_i, ...v_n\}$ so that for every edge (v_i, v_j) we have $i < j$.

There are several lemmas that can be inferred from the definition of a DAG. One lemma is: if G is a DAG, then G has a node with no incoming edges.

4.1 G is a DAG then G has a Topological Ordering [2 points]

Prove that if the graph G is a DAG, then G has a topological ordering.
Solution:
We prove the statement by induction on the number of nodes $n = |V|$.
Base case ($n = 1$): G consists of a single node v_1. It has no edges, since a self-loop (v_1, v_1) would be a directed cycle. The ordering $\{v_1\}$ is trivially a topological ordering.
Induction hypothesis: Every DAG with $n - 1$ nodes has a topological ordering.
Induction step: Let G be a DAG with n nodes.

1. By the lemma, G has a node v with no incoming edges.
2. Remove v together with all its outgoing edges to obtain $G' = G - \{v\}$. G' is still a DAG, because removing nodes and edges cannot create a directed cycle.
3. G' has $n - 1$ nodes, so by the induction hypothesis it has a topological ordering $u_1, \ldots, u_{n-1}$.
4. We define the ordering of G as $v, u_1, \ldots, u_{n-1}$. This is a topological ordering, since every edge points forward:
   - edges leaving v point forward, as v is placed first;
   - there are no edges entering v, as v has no incoming edges;
   - all edges between nodes of G' point forward by the induction hypothesis.

Hence every DAG has a topological ordering. ■

4.2 G has a Topological Ordering then G is a DAG [2 points]

Prove that if the graph G has a topological order, then G is a DAG.
Solution:
Proof by contradiction. Assume G has a topological ordering $v_1,\dots,v_n$, but G is not a DAG, i.e. it contains a directed cycle

$$v_{i_1} \to v_{i_2} \to \cdots \to v_{i_k} \to v_{i_1}.$$

By the definition of a topological ordering, every edge (v_i, v_j) satisfies $i < j$. Applying this to every edge of the cycle gives

$$i_1 < i_2 < \cdots < i_k < i_1,$$

and hence $i_1 < i_1$, which is a contradiction. Therefore G contains no directed cycle, i.e. G is a DAG. ■

4.3 Computational Graphs and Backpropagation [6 points]

For the given expression:

$$A = 2 \cdot ((a+b) \cdot (a-b)) \tag{9}$$

We can introduce intermediate variables such that there is a single mathematical operation per assignment such as:

$$t_1 = a+b \qquad t_2 = a-b \qquad t_3 = t_1 \cdot t_2 \qquad A = 2 \cdot t_3 \tag{10}$$

We can then use that to draw the corresponding computation graph based on the introduced variables as shown in Figure 2:

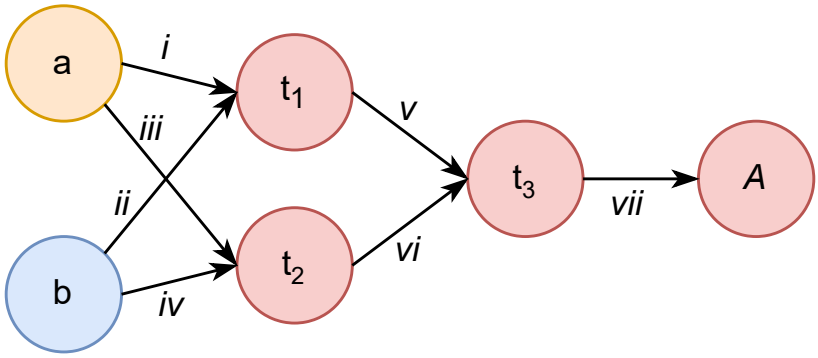


Figure 2: Computation Graph based on the introduced intermediate variables.

Having the computation graph, we can write down the products of partial derivatives that are computed by a node during backpropagation for all of our nodes. Indicating to which node the results are sent to we can write:

$$\begin{aligned} vii &:= \frac{\partial A}{\partial A}\frac{\partial A}{\partial t_3} \quad vi := \frac{\partial A}{\partial t_3}\frac{\partial t_3}{\partial t_2} \quad v := \frac{\partial A}{\partial t_3}\frac{\partial t_3}{\partial t_1} \\ iv &:= \frac{\partial A}{\partial t_2}\frac{\partial t_2}{\partial b} \quad iii := \frac{\partial A}{\partial t_2}\frac{\partial t_2}{\partial a} \quad ii := \frac{\partial A}{\partial t_1}\frac{\partial t_1}{\partial b} \\ i &:= \frac{\partial A}{\partial t_1}\frac{\partial t_1}{\partial a} \end{aligned} \tag{11}$$

With this we can now write down the sum of partial derivative products that calculates the derivative of the output node w.r.t. to this node for all of our nodes as:

$$A: \frac{\partial A}{\partial A} \qquad t_3: \frac{\partial A}{\partial t_3} \qquad t_2: \frac{\partial A}{\partial t_2} \qquad t_1: \frac{\partial A}{\partial t_1}$$

$$b: \frac{\partial A}{\partial t_2}\frac{\partial t_2}{\partial b} + \frac{\partial A}{\partial t_1}\frac{\partial t_1}{\partial b} \qquad a: \frac{\partial A}{\partial t_2}\frac{\partial t_2}{\partial a} + \frac{\partial A}{\partial t_1}\frac{\partial t_1}{\partial a} \tag{12}$$

Now it is your turn. Given is the expressions:

$$A = \frac{\mathrm{e}^{a}}{\sqrt{b}} \log\left(a + b\right) \tag{13}$$

a) Introduce intermediate variables (t_1, t_2, t_3, ...) such that there is a single mathematical operation per assignment. Reuse intermediate variables whenever possible.

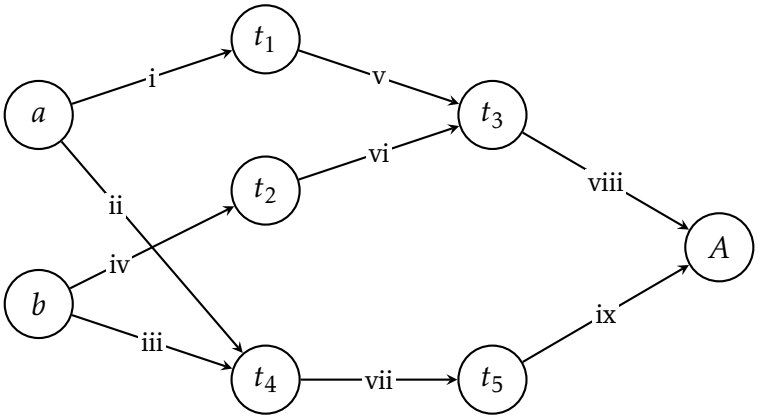
Solution:

$$t_1 = \mathrm{e}^{a} \qquad t_2 = \sqrt{b} \qquad t_3 = \frac{t_1}{t_2} \qquad t_4 = a + b \qquad t_5 = \log(t_4) \qquad A = t_3 \cdot t_5$$

The inputs are reused: a is used in t_1 and t_4, and b is used in t_2 and t_4.

b) Draw the corresponding computation graph based on the variables you introduced in a).

Solution:



c) For each edge in your computation graph, write down the products of partial derivatives $\left(\text{e.g. } \frac{\partial \mathcal{L}}{\partial y}\frac{\partial y}{\partial x}\right)$ that are computed by this edge during backpropagation. For each such product indicate to which node the results are sent to (e.g. by writing down the products close to the corresponding graph edges).
Note: you do not need to symbolically calculate or simplify the derivative for specific operations.

Solution:

Each edge multiplies the derivative arriving at its end node with its local derivative and sends the result to its start node ("edge → receiving node: product"):

$$\text{ix} \to t_5: \frac{\partial A}{\partial A}\frac{\partial A}{\partial t_5} \qquad \text{viii} \to t_3: \frac{\partial A}{\partial A}\frac{\partial A}{\partial t_3} \qquad \text{vii} \to t_4: \frac{\partial A}{\partial t_5}\frac{\partial t_5}{\partial t_4}$$

$$\text{vi} \to t_2: \frac{\partial A}{\partial t_3}\frac{\partial t_3}{\partial t_2} \qquad \text{v} \to t_1: \frac{\partial A}{\partial t_3}\frac{\partial t_3}{\partial t_1} \qquad \text{iv} \to b: \frac{\partial A}{\partial t_2}\frac{\partial t_2}{\partial b}$$

$$\text{iii} \to b: \frac{\partial A}{\partial t_4}\frac{\partial t_4}{\partial b} \qquad \text{ii} \to a: \frac{\partial A}{\partial t_4}\frac{\partial t_4}{\partial a} \qquad \text{i} \to a: \frac{\partial A}{\partial t_1}\frac{\partial t_1}{\partial a}$$

d) At each node, write down the sum of partial derivative products that calculates the derivative of the output node w.r.t. to this node. Your expressions only need to contain the direct children for the node and again you do not need to symbolically calculate or simplify the derivatives.

Solution:
Each node sums the products it receives from its direct children:

$$A: \frac{\partial A}{\partial A} \qquad t_5: \frac{\partial A}{\partial A}\frac{\partial A}{\partial t_5} \qquad t_3: \frac{\partial A}{\partial A}\frac{\partial A}{\partial t_3}$$

$$t_4: \frac{\partial A}{\partial t_5}\frac{\partial t_5}{\partial t_4} \qquad t_2: \frac{\partial A}{\partial t_3}\frac{\partial t_3}{\partial t_2} \qquad t_1: \frac{\partial A}{\partial t_3}\frac{\partial t_3}{\partial t_1}$$

$$b: \frac{\partial A}{\partial t_2}\frac{\partial t_2}{\partial b} + \frac{\partial A}{\partial t_4}\frac{\partial t_4}{\partial b} \qquad a: \frac{\partial A}{\partial t_1}\frac{\partial t_1}{\partial a} + \frac{\partial A}{\partial t_4}\frac{\partial t_4}{\partial a}$$

Only a and b have two children (t_1, t_4 and t_2, t_4 respectively), so only their expressions consist of a sum of two products; all other nodes have exactly one child.

5 Logistic Regression [5 points]

Using the chain rule, show that:

$$\nabla_{\mathbf{w}} \frac{1}{B} \sum_{i=1}^{B} \mathcal{L}(y_i, \hat{y}_i) = \frac{1}{B} \sum_{i=1}^{B} (\hat{y}_i - y_i)\, \mathbf{x}_i \tag{14}$$

where

$$\begin{aligned} \mathcal{L}(y_i, \hat{y}_i) &= -y \log(\hat{y}) - (1-y) \log(1-\hat{y}) \\ \hat{y}_i &= \sigma(\mathbf{w}^\intercal \mathbf{x}_i) \end{aligned} \tag{15}$$

Solution:
Since the gradient is linear, it can be moved inside the sum:

$$\nabla_w \frac{1}{B} \sum_{i=1}^{B} L(y_i, \hat{y}_i) = \frac{1}{B} \sum_{i=1}^{B} \nabla_w L(y_i, \hat{y}_i).$$

So it suffices to compute the gradient for a single sample i. Let $z_i = w^\top x_i$, so that $\hat{y}_i = \sigma(z_i)$. The loss depends on w via the chain $w \to z_i \to \hat{y}_i \to L$, hence by the chain rule

$$\nabla_w L(y_i, \hat{y}_i) = \frac{\partial L}{\partial \hat{y}_i} \cdot \frac{\partial \hat{y}_i}{\partial z_i} \cdot \nabla_w z_i.$$

We compute the three factors separately.
1. Derivative of the loss w.r.t. the prediction:

$$\frac{\partial L}{\partial \hat{y}_i} = -\frac{y_i}{\hat{y}_i} + \frac{1-y_i}{1-\hat{y}_i} = \frac{-y_i(1-\hat{y}_i) + (1-y_i)\hat{y}_i}{\hat{y}_i(1-\hat{y}_i)} = \frac{\hat{y}_i - y_i}{\hat{y}_i(1-\hat{y}_i)}.$$

2. Derivative of the sigmoid (see task 1.2 a):

$$\frac{\partial \hat{y}_i}{\partial z_i} = \sigma(z_i)\Big(1 - \sigma(z_i)\Big) = \hat{y}_i(1-\hat{y}_i).$$

3. Gradient of the linear part: since $z_i = w^\top x_i = \sum_j w_j x_{ij}$, we have $\frac{\partial z_i}{\partial w_j} = x_{ij}$, i.e.

$$\nabla_w z_i = x_i.$$

Multiplying the three factors, the term $\hat{y}_i(1-\hat{y}_i)$ cancels:

$$\nabla_w L(y_i, \hat{y}_i) = \frac{\hat{y}_i - y_i}{\hat{y}_i(1-\hat{y}_i)} \cdot \hat{y}_i(1-\hat{y}_i) \cdot x_i = (\hat{y}_i - y_i)\, x_i.$$

Inserting this into the averaged sum gives

$$\nabla_w \frac{1}{B} \sum_{i=1}^{B} L(y_i, \hat{y}_i) = \frac{1}{B} \sum_{i=1}^{B} (\hat{y}_i - y_i)\, x_i. \qquad \blacksquare$$